AMERICAN LATIN CLASS ATTENDANCE SYSTEM

Aws Deployment Guide

Table of Contents

AWS Deployment Guide

Do not use Render or Netlify. The target deployment uses EC2 plus Amazon RDS PostgreSQL.

Services

Service	Port	Responsibility
Frontend EC2 + Nginx	80/443	Serve landing page and private web app static files.
Core Business API EC2	3000	Branches, students, teachers, classes, schedules, attendance.
Auth & Session API EC2	3001	Google OAuth, sessions, refresh/session revocation, roles.
Reports & Rules API EC2	3002	Reports, scholarship candidates, promotion candidates, teacher hours/payment.

Service	Port	Responsibility
Amazon RDS PostgreSQL	5432	Normalized relational database.

The current academic executable is one Express codebase. For AWS, run the same codebase as separate Node processes with route ownership documented above, or split by route module if the course requires physical service separation.

Network

- VPC with public subnets for ALB/Nginx.
- Private app subnets for API EC2 instances.
- Private database subnets for RDS.
- Optional Application Load Balancer terminates HTTPS with ACM certificate.

Security Groups

Group	Inbound
ALB	`443` from internet, optional `80` redirect to HTTPS.
Frontend EC2	`80/443` only from ALB.
API EC2	`3000/3001/3002` only from ALB or frontend security group.
RDS	`5432` only from API security groups.

Environment Variables

- `NODE_ENV=production`
- `DB_DRIVER=prisma`
- `DATABASE_URL=postgres://...`
- `SESSION_SECRET`
- `SESSION_TTL_MINUTES`
- `GOOGLE_CLIENT_ID`
- `ALLOW_MOCK_GOOGLE_TOKENS=false`
- `CORS_ORIGINS=https://app.americanlatinclass.edu`
- `AUTH_RATE_LIMIT_MAX=20`
- `AWS_REGION`

Store secrets in AWS Systems Manager Parameter Store or Secrets Manager.

Deployment Steps

1. Create RDS PostgreSQL and run `migrations/001_initial_schema.sql`.
2. Run `seeders/001_seed.sql`.
3. Install Node.js LTS on EC2 instances.
4. Copy project or deploy from repository.
5. Run `npm ci --omit=dev` on production instances.
6. Start each Node process with its assigned port.
7. Configure Nginx reverse proxy or ALB target groups.
8. Enable HTTPS and configure HSTS at ALB/Nginx.
9. Verify `/api/v1/auth/me`, `/api/v1/branches`, reports and private page redirects.

HTTPS Recommendation

Use ACM certificates on ALB. Redirect all HTTP traffic to HTTPS. Keep cookies Secure in production.

Google OAuth Origins

For Google Sign-In, configure Authorized JavaScript origins with an HTTPS public domain:

- Local development: `http://localhost:5500`, `http://127.0.0.1:5500`
- Production: `https://app.americanlatinclass.edu` or another domain controlled by the team

Do not use raw public IP origins such as `http://18.217.255.109`; Google rejects them because they do not end with a public top-level domain. If the team uses a temporary DNS name, configure HTTPS first and add the exact origin used by the browser.

React Frontend Build on EC2

Before starting the production Express process, build the React + Vite frontend:

```
cd 06Code
npm ci
npm run db:generate
npm run frontend:build
npm start
```

The Vite output is `06Code/dist/frontend`. Express serves that directory and keeps `/api/v1` as the API prefix. If Nginx is used, configure the site root to `06Code/dist/frontend` and proxy `/api/v1` to the Node.js process.

For the current EC2 frontend instance, the reference Nginx configuration is versioned at `07Other/nginx-alc-frontend.conf`. It includes:

- `/api/v1/auth/` proxy to Auth & Session API.
- `/api/v1/reports/` proxy to Reports & Rules API.
- `/api/v1/` proxy to Core Business API.
- `/private/` no-cookie redirect to `/login.html?session=expired` plus `Cache-Control: no-store`.
- Temporary HTTPS host `https://18-217-255-109.sslip.io` for Google OAuth validation.